

Anexo C — Palabras reservadas, símbolos y léxico

Listado exhaustivo de tokens por lenguaje, derivado de las gramáticas (`src/main/antlr4/`). El sombreado de la interfaz clasifica cada token en las categorías KEYWORD, TYPE, BOOLEAN, NUMBER, STRING, CHAR, COMMENT, OPERATOR, PUNCTUATION, IDENTIFIER (ver `HighlightService`).

C.1 Lat — Pig Latin (.pig)

C.1.1 Palabras de sección y estructurales

Token	Lexema	Uso
VARIABLES_SECTION	VARIABLES>	Inicio de la sección opcional de variables globales
FUNCTIONS_SECTION	MUNERA>	Inicio de la sección opcional de funciones
MAIN_SECTION	MAIOR>	Inicio de la sección obligatoria principal
FINIS_PROGRAM	FINIS	Cierre del programa (se usa FINIS;)
FINIS	finis	Cierre de bloques si, dum y de funciones
ESTO	esto	Declaración de variable / parámetro
SERIES	series	Declaración de arreglo
ACTIO	actio	Función sin retorno
RATIO	ratio	Función con retorno
IMPORT	import	Importación de módulo .y/.z
NOVUS	novus	Creación de objeto

Token	Lexema	Uso
SI / ALITER	si / aliter	Condicional / siguientes ramas
DUM	dum	Ciclo mientras
FACERE	facere	Ciclo hacer-mientras
PER	per	Ciclo con iterador
REDDERE	reddere	Retorno
PERGE	perge	Continuar
INTERRUMPE	interrumpe	Romper
VARIABLES	VARIABLES	Palabra de la sección local dentro de funciones

C.1.2 Tipos y booleanos

Token	Lexema	TypeKind
NUMERUS	numerus	INT
DECIMALIS	decimalis	DOUBLE
TEXTUM	textum	STRING
LITTERA	littera	CHAR
BOOL	bool	BOOLEAN
VERUM	verum	literal true
FALSUS	falsus	literal false
ID	(identificador)	tipo struct/clase definido por el usuario

C.1.3 Operadores y símbolos

Token	Lexema	Categoría
PLUS / MINUS / MULT / DIV	+ - * /	Operador aritmético

Token	Lexema	Categoría
PLUSPLUS / MINUSMINUS	++ --	Operador incremento/decremento
AND / OR / NON	&& non	Operador lógico
EQUAL / NOT_EQUAL	== !=	Relacional
LESS / GREATER / LESS_EQUAL / GREATER_EQUAL	< > <= >=	Relacional
PRINT	>>	Imprimir
READ	<<	Leer
= ; , : () { } [] .	—	Puntuación
NUMBER	[0-9]+	Número entero
DECIMAL	[0-9]+.[0-9]+	Decimal
STRING	"..."	Cadena (sin saltos de línea)
CHAR	'x'	Carácter
LINE_COMMENT	// ...	Comentario (oculto)
BLOCK_COMMENT	## ... ##	Comentario de bloque (oculto)
WS	espacios/tabs/saltos	Ignorado

C.2 Y? (.y)

C.2.1 Palabras reservadas

Token	Lexema	Uso
STRUCTURES	%estructuras	Sección (opcional) de estructuras

Token	Lexema	Uso
FUNCTIONS	%funciones	Sección (obligatoria) de funciones
STRUCTURE	estructura	Definición de estructura
DEFINE	definir	Definición de función
IF / ELSE / OTHERWISE	si / sino / contrario	Condicional
THEN	entonces	Une condición con bloque
SWITCH / CASE / ALWAYS	elegir / caso / siempre	Selección múltiple
FOR / WHILE / DO	para / mientras / hacer	Ciclos
BREAK / CONTINUE	romper / continuar	Control de ciclo
RETURN	retornar	Retorno
PRINT / READ	imprimir / leer	E/S del sistema
TRUE / FALSE	verdadero / falso	Booleanos

C.2.2 Tipos

Token	Lexema	TypeKind
INTEGER	entero	INT
FLOAT	flotante	DOUBLE
CHARACTER	caracter	CHAR
BOOLEAN	bool	BOOLEAN
STRING	cadena	STRING
ID	(identificador)	STRUCT definido por el usuario

C.2.3 Operadores y símbolos

Token	Lexema	Categoría
INCREMENT / DECREMENT	++ --	Incremento/decremento
PLUS / MINUS / MULT / DIV / MOD	+ - * / %	Aritmético
AND / OR / NOT	&& !	Lógico
EQUAL / NOT_EQUAL	== !=	Relacional
LESS / GREATER / LESS_EQUAL / GREATER_EQUAL	< > <= >=	Relacional
PLUS_ASSIGN / MINUS_ASSIGN / MULT_ASSIGN / ASSIGN	+= -= *= =	Asignación
ARROW	->	Tipo de retorno
: ; , . () [] { }	—	Puntuación
FLOAT_LITERAL	[0-9]+.[0-9]+	Decimal
INTEGER_LITERAL	[0-9]+	Entero
CHAR_LITERAL	'x'	Carácter
STRING_LITERAL	"..."	Cadena
INDENT / DEDENT	(sintéticos)	Producidos por IndentationTokenSource
NEWLINE / WS	salto / espacios	Delimitadores
LINE_COMMENT / BLOCK_COMMENT	// ... / /* ... */	Comentarios (ocultos)

Nota: el % es MOD (módulo). El % de %estructuras/%funciones se reconoce primero como los tokens STRUCTURES/FUNCTIONS.

C.3 Zetariano (.z)

C.3.1 Palabras reservadas

Token	Lexema	Uso
PUBLIC / CLASS	public / class	Declaración de clase y miembros públicos
VOID	void	Método sin retorno
RETURN	return	Retorno
IF / ELSE	if / else	Condicional
SWITCH / CASE / DEFAULT	switch / case / default	Selección múltiple
FOR / WHILE / DO	for / while / do	Ciclos
BREAK / CONTINUE	break / continue	Control de ciclo
PRINT / PRINTLN / READLN	print / println / readln	E/S del sistema
NEW	new	Crear objeto/arreglo
NULL	null	Referencia nula
TRUE / FALSE	true / false	Booleanos

C.3.2 Tipos

Token	Lexema	TypeKind
INT	int	INT
DOUBLE	double	DOUBLE
CHAR	char	CHAR
BOOLEAN	boolean	BOOLEAN
STRING	String	STRING
ID	(identificador)	CLASS del usuario

C.3.3 Operadores y símbolos

Token	Lexema	Categoría
PLUS / MINUS / MULT / DIV / MOD	+ - * / %	Aritmético
INCREMENT / DECREMENT	++ --	Incremento/decremento
AND / OR / NOT	&& !	Lógico
EQUAL / NOT_EQUAL	== !=	Relacional
LESS / GREATER / LESS_EQUAL / GREATER_EQUAL	< > <= >=	Relacional
ASSIGN / PLUS_ASSIGN / MINUS_ASSIGN / MULT_ASSIGN	= += -= *=	Asignación
QUESTION	?	Ternario (junto con :)
() { } [] . , : ;	—	Puntuación
FLOAT_LITERAL	[0-9]+.[0-9]+	Decimal
INTEGER_LITERAL	[0-9]+	Entero
CHAR_LITERAL	'x' o '\x'	Carácter (admite escape)
STRING_LITERAL	"..." con escapes	Cadena
LINE_COMMENT / BLOCK_COMMENT	// ... /* ... */	Comentarios (ocultos)
WS	espacios/saltos	Ignorado

C.4 Identificadores

Los tres lenguajes usan la misma forma de identificador:

ID : [a-zA-Z][a-zA-Z0-9_]*

- Lat y Y son **sensibles a mayúsculas**.
- Z también distingue mayúsculas/minúsculas.
- Las palabras reservadas tienen prioridad sobre ID (aparecen antes en la gramática del lexer).

C.5 Literales de cadena y carácter

Lenguaje	Cadena	Carácter	Escapes
Lat	"texto" (sin " ni salto)	' x ' (cualquier carácter)	No
Y	"texto" (sin " ni salto)	' x ' (cualquier carácter)	No
Z	"texto" (admite \ + carácter)	' x ' o '\ x '	Sí (\ ", \ \, etc.)